

Laboratorio Semana 1

Objetivo

Aplicar en un solo flujo de trabajo lo visto en la semana: entorno reproducible con **uv**, colaboración con **Git y GitHub** usando ramas y pull requests, exploración y limpieza de un dataset incompleto con **Pandas**, análisis vectorizado con **NumPy** y pruebas automáticas con **pytest**.

Reglas

- El trabajo es en parejas. Uno es **Persona A** y el otro **Persona B**. Déjenlo escrito en el README.
- Nadie hace commit directo a main. Todo entra por Pull Request revisado por el compañero.
- Cada persona trabaja solo en el archivo que le toca. La única excepción es el README.
- `.venv/` y los datos crudos no se suben al repositorio.

Si te toca trabajar solo:

haces las **dos ramas** tú mismo, con los mismos nombres y el mismo reparto de archivos. Abres los dos Pull Requests y los mergeas tú. El conflicto en `## Hallazgos` se produce igual, porque escribes en la misma sección desde las dos ramas. Lo único que no aplica es la revisión cruzada: en su lugar, deja en cada PR un comentario en *Files changed* explicando una decisión que tomaste.

Se entrega al final:

el link del repositorio público y cuatro capturas de pantalla: (1) los dos Pull Requests mergeados, (2) el conflicto de merge antes de resolverlo, (3) `uv run pytest` en verde, (4) `git log --oneline --graph --all`.

0 Elegir el dataset

El dataset lo eligen ustedes. Solo tiene que cumplir dos condiciones:

- Servir para una tarea de **regresión** o de **clasificación**: definan cuál es la variable objetivo.
- Tener **datos faltantes**, para que haya algo real que explorar y limpiar.

Ojo con esto:

los faltantes casi nunca llegan como NaN. Pueden venir como `?`, como el centinela `-200`, como `unknown`, como cadena vacía o como un cero imposible (una persona de 0 años). Descubrir cómo están codificados es parte del trabajo y es lo primero que deben mirar.

Pueden buscar en cualquiera de estos dos repositorios:

- UCI Machine Learning Repository — <https://archive.ics.uci.edu/datasets>
- Kaggle Datasets — <https://www.kaggle.com/datasets>

Opciones ya verificadas. Los links cargan directo con `pd.read_csv(url)`:

Dataset	Tamaño	Tarea / objetivo	Cómo vienen los faltantes
Adult / Census Income archive.ics.uci.edu/static/public/2/d ata.csv	48842 x 15	Clasificación income	2203 celdas vacías en <code>workclass</code> , <code>occupation</code> , <code>native-country</code>
Communities and Crime archive.ics.uci.edu/static/public/183 /data.csv	1994 x 128	Regresión ViolentCrimesPerPo p	39202 faltantes marcados con <code>?</code> repartidos en 25 columnas
Air Quality archive.ics.uci.edu/static/public/360 /data.csv	9357 x 15	Regresión C6H6 (GT)	16701 celdas con el centinela <code>-200</code> en 13 columnas

Dataset	Tamaño	Tarea / objetivo	Cómo vienen los faltantes
Titanic raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv	891 x 12	Clasificación Survived	866 faltantes: Age (177), Cabin (687), Embarked (2)
Heart Disease (Cleveland) archive.ics.uci.edu/static/public/45/data.csv	303 x 14	Clasificación num	Solo 6 faltantes (ca, thal)
Auto MPG archive.ics.uci.edu/static/public/9/data.csv	398 x 9	Regresión mpg	Solo 6 faltantes en horsepower

Los dos últimos tienen muy pocos faltantes, así que dan menos material de limpieza que los tres primeros.

Registren en el README: nombre del dataset, link, tarea, variable objetivo y una frase de por qué lo eligieron.

1 Persona A crea el repositorio

```
# 1) Crear el proyecto con uv
uv init lab-semana1 && cd lab-semana1

# 2) Dependencias del proyecto y de desarrollo
uv add pandas numpy matplotlib pyarrow
uv add --dev pytest ruff

# 3) Estructura de carpetas
mkdir src tests data
```

Crear el archivo `.gitignore` en la raíz del proyecto (al lado de `pyproject.toml`, NO dentro de `.venv/`):

```
.venv/
__pycache__/
.pytest_cache/
data/*.csv
data/*.parquet
```

Crear el `README.md` con esta plantilla. Respeten los encabezados tal cual: importan para la Etapa 4.

```
# Lab Semana 1 - <nombre del dataset>

- Persona A: <nombre>
- Persona B: <nombre>
- Dataset: <link>
- Tarea: <regresion | clasificacion>   Variable objetivo: <columna>

## Como correr

    uv sync
    uv run pytest -q
    uv run python main.py

## Hallazgos

## Decisiones de limpieza
```

Primer commit y publicación en GitHub:

```
git init
git add .
git commit -m "chore: proyecto base con uv, estructura y README"
git branch -M main
```

```
# Crear en github.com un repositorio VACIO (sin README, sin .gitignore, sin licencia)
git remote add origin https://github.com/<usuarioA>/lab-semana1.git
git push -u origin main
```

Invitar al compañero:

en GitHub, **Settings** → **Collaborators** → **Add people** e invitar a la Persona B. B tiene que aceptar la invitación desde su correo o desde github.com/notifications; si no acepta, no va a poder hacer push.

2 Persona B clona el repositorio

```
git clone https://github.com/<usuarioA>/lab-semana1.git
cd lab-semana1

uv sync          # reconstruye .venv exactamente desde uv.lock
uv run python -c "import pandas, numpy; print('ok', pandas.__version__)"
```

Pregunta de investigación 1 (respondan en el README):

¿por qué `uv sync` puede reconstruir el entorno aunque `.venv/` no esté versionado en el repositorio? ¿Qué archivo se lo permite y qué guarda exactamente ese archivo? Dos líneas bastan.

3 Cada uno en su propia rama

Por qué funciona esta división:

las dos personas del equipo pueden programar al mismo tiempo. Para que B no tenga que esperar a que A termine de limpiar, **las funciones de B reciben un DataFrame como argumento** en lugar de leer el archivo que produce A. Así B prueba su código con un DataFrame inventado de 4 filas.

Persona A — rama feature/carga-limpieza

```
git switch -c feature/carga-limpieza
```

Escribir `src/carga.py` con estas cuatro funciones:

Función	Qué debe hacer
<code>cargar(url, na_values=None)</code>	<code>pd.read_csv</code> desde la URL. El parámetro <code>na_values</code> es lo que les permite decirle a pandas que <code>'?'</code> o <code>-200</code> son faltantes desde la lectura.
<code>reporte_nulos(df)</code>	Devuelve un DataFrame con una fila por columna: conteo de nulos y porcentaje, ordenado de mayor a menor.
<code>limpiar(df)</code>	En este orden: <code>replace([np.inf, -np.inf], np.nan)</code> , <code>drop_duplicates()</code> , <code>.str.strip().str.lower()</code> en las columnas de texto, y tratar los faltantes columna por columna. Cierra con <code>reset_index(drop=True)</code> .
<code>guardar(df, ruta)</code>	Guardar el resultado en formato Parquet con <code>to_parquet</code> .

No hacer `dropna()` sin criterio.

Si una columna tiene 80% de nulos, lo que se elimina es **la columna**, no las filas. Si una columna numérica tiene 3% de nulos, probablemente conviene imputar con la mediana. Cada decisión que tomen va escrita en la sección *Decisiones de limpieza* del README, con su justificación.

Cerrar con un bloque `if __name__ == "__main__":` que ejecute el flujo completo y deje `data/limpio.parquet` escrito.

Persona B — rama feature/analysis

```
git switch -c feature/analysis
```

Escribir `src/analysis.py` con estas cinco funciones. **Todas reciben los datos como argumento y ninguna abre archivos.**

Función	Qué debe hacer
<code>filtrar(df, columna, umbral)</code>	Máscara booleana: devuelve solo las filas donde <code>df[columna] > umbral</code> .
<code>resumen_por_grupo(df, col_grupo, cols_num)</code>	<code>groupby(col_grupo)[cols_num].agg(['mean', 'std', 'count'])</code> , para comparar el comportamiento entre categorías.
<code>zscore(matriz)</code>	Recibe un <code>np.ndarray</code> 2D y lo normaliza por columna usando broadcasting: <code>(m - m.mean(axis=0)) / m.std(axis=0)</code> . Prohibido usar un bucle for.
<code>top_k(df, columna, k)</code>	Los <code>k</code> registros con mayor valor en esa columna, usando <code>np.argsort</code> .
<code>recta_minimos_cuadrados(x, y)</code>	Ajusta <code>y = a*x + b</code> con <code>np.linalg.lstsq</code> y devuelve la tupla <code>(a, b)</code> . Pista: hay que armar la matriz de diseño apilando <code>x</code> junto a una columna de unos.

T Como armar las pruebas con pytest

Una prueba automática es una función normal de Python que llama a su código con una entrada conocida y verifica que la salida sea la esperada. Si la verificación falla, `pytest` lo reporta. No hay que instalar nada extra: ya lo agregaron con `uv add --dev pytest`.

Las tres reglas que pytest necesita para encontrar sus pruebas

- El archivo debe llamarse `test_algo.py` o `algo_test.py`, y vivir en la carpeta `tests/`.
- Cada prueba es una función cuyo nombre empieza con `test_`.
- Dentro se usa `assert`: si la expresión es verdadera la prueba pasa, si es falsa la prueba falla.

La estructura de una prueba (preparar, ejecutar, verificar)

```
# tests/test_ejemplo.py
from src.analisis import filtrar
import pandas as pd

def test_filtrar_deja_solo_los_mayores():
    # 1. PREPARAR: datos de entrada que ustedes controlan
    df = pd.DataFrame({"edad": [10, 25, 40]})

    # 2. EJECUTAR: llamar a la funcion que se quiere probar
    resultado = filtrar(df, "edad", 20)

    # 3. VERIFICAR: comparar contra lo que deberia salir
    assert len(resultado) == 2
    assert resultado["edad"].min() > 20
```

El fixture: datos de prueba reutilizables

Si varias pruebas necesitan el mismo `DataFrame` de ejemplo, en vez de copiarlo en cada una se define una vez con el decorador `@pytest.fixture`. Después se pide poniendo el nombre del fixture como parámetro de la función de prueba, y `pytest` lo entrega automáticamente:

```
# tests/test_analisis.py
import numpy as np
import pandas as pd
import pytest
from src.analisis import zscore, resumen_por_grupo
```

```
@pytest.fixture                                # <- se define una sola vez
def df_mini():
    return pd.DataFrame({
        "grupo": ["a", "a", "b", "b"],
        "valor": [10.0, 20.0, 30.0, 40.0],
    })

def test_zscore_tiene_media_cero(df_mini):        # <- pytest inyecta df_mini
    z = zscore(df_mini[["valor"]].to_numpy())
    assert z.mean() == pytest.approx(0.0, abs=1e-9)

def test_resumen_por_grupo_calcula_la_media(df_mini):
    r = resumen_por_grupo(df_mini, "grupo", ["valor"])
    assert r.loc["a", ("valor", "mean")] == pytest.approx(15.0)
```

Por qué `pytest.approx` y no `==`:

los float casi nunca son exactamente iguales por errores de redondeo binario. De hecho `0.1 + 0.2 == 0.3` da `False` en Python. Para comparar arreglos completos de NumPy usen `np.testing.assert_allclose(a, b)`.

Ideas de qué probar en este laboratorio

Función	Qué verificar
<code>limpiar(df)</code>	Que un DataFrame con dos filas idénticas quede con una sola fila después de limpiar.
<code>limpiar(df)</code>	Que después de limpiar no quede ningún <code>inf</code> ni ningún nulo en las columnas numéricas.
<code>filtrar(df, col, u)</code>	Que el número de filas devueltas sea el que ustedes contaron a mano.
<code>zscore(m)</code>	Que cada columna del resultado tenga media 0 y desviación 1.
<code>recta_minimos_cuadrados</code>	Que con puntos perfectamente alineados ($y = 2x + 1$) devuelva $a=2$ y $b=1$.

Correr las pruebas y leer el resultado

```
uv run pytest                # salida completa
uv run pytest -q             # salida resumida
uv run pytest -v             # nombre de cada prueba, una por línea
uv run pytest tests/test_analisis.py::test_zscore_tiene_media_cero # una sola
```

En la salida, cada `.` es una prueba que pasó y cada `F` una que falló. Cuando algo falla, `pytest` imprime la línea exacta del `assert` y los valores que comparó, así que ahí está la pista de qué arreglar.

Antes de hacer push, los dos:

```
uv run pytest -q             # las pruebas deben pasar
uvx ruff check .             # sin errores de estilo

git add .
git commit -m "feat: <lo que hiciste>"
git push -u origin <tu-rama>
```

Pregunta de investigación 2 (respondan en el README):

¿qué diferencia hay entre correr `pytest` a secas y `uv run pytest`? Pista: tiene que ver con **cuál** Python y **cuál** entorno terminan ejecutando las pruebas, y con qué pasa si alguien no activó el entorno virtual.

4 Los dos escriben en el README

Todavía dentro de su propia rama, **cada uno** agrega su hallazgo principal justo debajo del encabezado **## Hallazgos del README**, en la misma posición:

```
# Persona A escribe esto en su rama:
## Hallazgos
- (A) <tu hallazgo sobre la calidad de los datos>

# Persona B escribe esto en la suya:
## Hallazgos
- (B) <tu hallazgo del analisis>
```

Cada uno hace commit y push en su rama.

Esto va a generar un conflicto, y es intencional.

Los dos modificaron la misma línea del mismo archivo en ramas distintas. Git no tiene forma de decidir cuál versión vale, así que en la Etapa 5 van a tener un conflicto de merge real. Resolverlo es parte del laboratorio.

5 Pull Requests cruzados y el conflicto

1. En GitHub, cada uno abre un Pull Request de su rama hacia main (aparece el botón *Compare & pull request*). Título descriptivo y tres líneas de descripción: qué hiciste y por qué.
2. **Revísense cruzado:** A revisa el PR de B y B revisa el de A. Cada revisor debe dejar al menos un comentario en la pestaña *Files changed* (clic en el número de línea y escribir el comentario) y después aprobar el PR. El comentario tiene que decir algo concreto del código.
3. Mergean el **primer** PR. Ese entra sin problemas.
4. Al volver al segundo PR, GitHub va a mostrar *"This branch has conflicts that must be resolved"*. Se resuelve desde la terminal:

```
git switch <la-rama-del-segundo-PR>
git fetch origin
git merge origin/main          # aqui aparece el conflicto
```

Git deja el archivo marcado así:

```
## Hallazgos
<<<<<<< HEAD
- (B) hallazgo de B
=====
- (A) hallazgo de A
>>>>>>> origin/main
```

Editén `README.md` a mano dejando **los dos** hallazgos y borrando las tres líneas de marcadores (`<<<<<<<`, `=====`, `>>>>>>>`). El resultado debe quedar así:

```
## Hallazgos
- (A) hallazgo de A
- (B) hallazgo de B
```

```
git add README.md
git commit -m "merge: resuelve conflicto en la seccion Hallazgos"
git push
```

El PR se desbloquea solo. Mergeenlo.

Entregable de esta etapa:

la **captura de pantalla del conflicto antes de resolverlo**: la terminal con el mensaje `CONFLICT (content): Merge conflict in README.md`, o el archivo con los marcadores abiertos en el editor.

6 Integración final en main

```
git switch main
git pull
uv sync          # por si el otro agrego alguna dependencia
```

Crear `main.py` en la raíz, que importe **los dos módulos** y produzca, en este orden:

- El reporte de nulos del dataset crudo, antes de limpiar.
- Un `resumen_por_grupo` sobre el dataset ya limpio.
- Los `top_k` con `k=5` de alguna columna que tenga sentido para su pregunta.
- La recta de mínimos cuadrados entre dos variables numéricas, imprimiendo la pendiente y el intercepto.
- Un gráfico guardado con `plt.savefig("figura.png")`.

Verificación final antes de entregar:

```
uv run pytest -q          # todo en verde
uv run python main.py     # corre de principio a fin sin errores
git log --oneline --graph --all # se ven las dos ramas y los dos merges
```

Y completar el README: los hallazgos, la sección *Decisiones de limpieza* (qué imputaron, qué eliminaron y por qué) y las dos preguntas de investigación.

! Errores comunes

Síntoma	Por qué pasa	Cómo se arregla
<code>ModuleNotFoundError: No module named 'src'</code> al correr <code>pytest</code>	<code>pytest</code> no sabe que la raíz del proyecto es parte del path de importación	Crear un <code>src/__init__.py</code> vacío y añadir a <code>pyproject.toml</code> : [tool.pytest.ini_options] pythonpath = ["."]
<code>git push</code> rechazado: <i>Updates were rejected</i>	El remoto tiene commits que ustedes no tienen localmente	<code>git pull --rebase</code> y volver a intentar el push
Al compañero le falla un <code>import</code> que a ti te funciona	Instalaste algo con <code>pip install</code> en vez de <code>uv add</code> , así que nunca quedó registrado en <code>pyproject.toml</code>	Siempre <code>uv add <paquete></code> . Commitear <code>pyproject.toml</code> y <code>uv.lock</code> ; el otro corre <code>uv sync</code>
Se subió <code>.venv/</code> al repositorio	El <code>.gitignore</code> quedó en el lugar equivocado o se creó después del primer <code>git add</code>	<code>git rm -r --cached .venv</code> y commit. El <code>.gitignore</code> va en la raíz , no dentro de <code>.venv/</code> (ese lo crea <code>uv</code> solo)
Windows: error de ruta demasiado larga	El límite histórico de 260 caracteres en las rutas de Windows	Clonar el repositorio en una ruta corta, tipo <code>C:\dev\</code>
El dataset "no tiene faltantes" pero los datos se ven raros	Los faltantes están codificados como <code>?</code> , <code>-200</code> o <code>unknown</code> , y <code>pandas</code> los leyó como texto o como número válido	Revisar <code>df.describe()</code> y <code>df[col].unique()</code> . Recargar con <code>pd.read_csv(url, na_values=["?", -200])</code>
<code>uv run pytest</code> dice <i>collected 0 items</i>	Los archivos o las funciones no siguen la convención de nombres	Los archivos deben llamarse <code>test_*.py</code> y las funciones <code>def test_*</code>
El conflicto "no se va" aunque ya editaron el archivo	Quedaron marcadores de conflicto sueltos en alguna parte del archivo	Buscar <code><<<<<<</code> en todo el archivo y borrar las tres líneas de marcadores; luego <code>git add + git commit</code>

OK Checklist de entrega

- ☐ Repositorio público en GitHub, con los dos como colaboradores.
- ☐ Dos ramas de trabajo, cada una con al menos **2 commits descriptivos**.
- ☐ Dos Pull Requests, cada uno con al menos un comentario visible en *Files changed* (revisado por el compañero, o por ti mismo si trabajaste solo).
- ☐ Un conflicto de merge resuelto, con su captura de pantalla.
- ☐ Al menos **2 pruebas** que pasan con `uv run pytest`.
- ☐ `main.py` corre de principio a fin e importa los dos módulos.
- ☐ README completo: dataset y objetivo, hallazgos de los dos, decisiones de limpieza justificadas y las dos preguntas de investigación respondidas.

Sobre el uso de asistentes de programación:

pueden apoyarse en herramientas de programación agentica como **Codex**, **Claude Code**, **Cursor**, **OpenCode** o **Antigravity** para escribir y depurar el código. Lo que sí se espera es que entiendan y puedan explicar cada línea del repositorio que entregan, sobre todo las decisiones de limpieza y las pruebas.